

论文模板 03 · 数据架构与混合存储设计

配套素材见 [00-项目素材库.md](#)。本主题近三年明显升温，且 BASE HOME 素材极佳。

一、真题常见问法

变体 A（数据架构整体）

请论述你参与的某系统的数据架构设计，包括数据模型设计、存储选型、数据一致性与数据治理等方面。

变体 B（存储选型）

请结合项目论述你在关系型数据库与 NoSQL 数据库之间的选型依据，以及如何保证多存储之间的一致性。

变体 C（大数据/高并发数据）

请论述你如何应对系统面临的海量数据存储与高并发访问挑战。

判断：变体 B 是 BASE HOME 的**最优命中区**——因为项目真实采用了 MySQL + DynamoDB + S3 三种存储，有真实选型依据可写。

二、摘要模板（约 300 字）

本文论述了本人参与的 H.I.S. BASE HOME 旅游业务平台的数据架构设计与实现。

该平台承载预约管理、客户消息、通知发送等业务，数据形态差异极大：

既有强一致的交易数据，也有字段频繁演进的半结构化数据与文件实体。

本人担任架构师，负责数据模型设计、存储选型与数据同步链路设计。

针对【单一存储难以兼顾强一致与弹性扩展】、【异构存储间一致性保障】、【批量作业与在线业务资源争抢】三大挑战，

本文从【混合存储选型】、【事件驱动同步】、【读写分离】三个维度进行设计：

以 Aurora MySQL 承载强一致交易（79 张业务与主数据表），

以 DynamoDB 承载高弹性半结构化数据（5 张表，单表多实体设计），

以 S3 承载文件实体，并通过 DynamoDB Stream 驱动 Serverless 链路完成最终一致同步。

系统上线后【 填效果】，验证了方案的可行性。

三、正文五段框架

第 1 段 · 项目背景与数据挑战（约 400 字）

核心是把「数据形态差异」讲清楚，这是选型的唯一依据。

旅游业务的数据形态差异极大，我将其归纳为四类：① **强一致交易数据**——预约、申请人、旅行者、费用，要求事务与关联查询；② **频繁演进的半结构化数据**——旅行主信息字段随业务政策频繁增删，若用关系表承载则每次变更都需 DDL；③ **文件实体**——护照、文档、亲权同意书，体积大且需对象存储能力；④ **高频读写的小数据**——消息线程、序列号，读写频繁且要求低延迟。用单一存储承载这四类数据，必然在某一方面做出妥协。

第 2 段 • 架构设计 (约 800 字, 本文核心)

第一层：混合存储选型 (Polyglot Persistence)

数据类型	存储选型	选型依据
强一致交易	Aurora MySQL	需要事务、外键、复杂关联查询；79 张表 (26 张 <code>M*</code> 主数据 + 53 张 <code>T*</code> 业务)
半结构化/弹性	DynamoDB 5 张表	schema-less、按需扩缩、毫秒级延迟，字段演进无需 DDL
文件实体	S3 (ap-northeast-1)	对象存储，与数据库解耦，降低库体积与备份成本
分布式序列号	DynamoDB <code>sequence</code> 表	需跨实例唯一且自增，关系库自增无法满足分布式场景

第二层：DynamoDB 单表多实体设计 (本项目的技术亮点)

我采用**分区键 + 排序键 + JSON 载荷**的模式实现单表多实体：以 `travel` 表为例，分区键为 `travel_no` (旅行编号)，排序键为 `data_type` (数据类型)，实体内容存于 `json_data` 字段。这样一张表可承载同一旅行下的多种数据类型，既避免了表数量随业务类型线性膨胀，又保证同一旅行的所有数据落在同一分区，**查询时可用一次 Query 取回**，无需跨分区扫描。

其余四张表：- `document` —— 文档元数据 (文件实体在 S3) - `parental` —— 亲权同意书 (親權同意書) - `message_thread` —— 消息线程 - `sequence` —— 序列号 (`seq_key` / `seq_no`)，为分布式环境提供唯一编号

治理措施：表名统一带环境前缀，实现「1 账户 1 集群」的环境隔离；`scan-limit` 设为 10000，为全表扫描设置硬边界，防止误用 Scan 拖垮表。

第三层：批处理读写分离

批处理既要大量读取 (成本文件生成、外部预约导入、ToDo 状态更新)，又要回写结果。我为其配置**双数据源**：`ReaderDataSourceConfig` (`spring.datasource.reader`) 与 `WriterDataSourceConfig` (`spring.datasource.writer`，标 `@Primary`)，通过包路径隔离 Mapper (`repository/reader/` 与 `repository/writer/`)，使读写请求物理分离，批量作业不再争抢在线业务的连接池资源。同时设置事务隔离级别为 `TRANSACTION_READ_COMMITTED`，避免长事务期间大量持有间隙锁影响在线写入；`defaultFetchSize = 200` 控制批量游标，防止一次性载入海量结果集导致 OOM。

第 3 段 • 关键实现 (约 700 字)

要素一：数据访问层设计

MySQL 侧统一使用 MyBatis (3.0.5, 在统一版本源中强制锁定, 覆盖 Spring Boot 托管版本以保证行为一致), 79 个实体按职责分两个 source set 组织, 主数据实体与业务实体分离; 54 个 Mapper 按业务域归入 repository/。DynamoDB 侧通过 DynamoDbEnhancedClient 提供对象映射, 由 DynamoDbRepository 基类封装通用操作, 实体以 @DynamoDbBean 注解声明, 属性名集中定义在 consts 包 (TravelConsts、DocumentConsts 等), 避免魔法字符串散落。

要素二：一致性与重试

DynamoDB 侧启用乐观锁重试 (@EnableRetry, 重试 4 次、间隔 500ms), 应对并发写入冲突。跨存储一致性通过事件驱动链路达成最终一致 (DynamoDB Stream → Lambda → MySQL)。

要素三：CSV 与文件下载

成本文件 (ACT 原价 / AIR 机票 / HTL 酒店) 使用 SuperCSV 注解库 将实体直接映射为 CSV, 配合 CsvDownloader 接口与 FileUploadService 落 S3, 避免大文件在内存中拼装。导出列定义通过注解集中在实体上, 新增列不会波及导出逻辑。

要素四：主数据治理

集团 MDM 主数据由 ImportMdmJob 定时同步, 本地保留 26 张 M* 表作为只读副本, 业务查询不直连外部系统——既保证主数据一致性, 又隔离了外部系统故障对业务的影响。

第 4 段 • 难点与解决 (约 700 字)

数据架构主题最佳组合：

优先级	难点	适配理由
首选	⑤ DynamoDB 单表多实体 + 混合存储	主题核心
次选	④ MySQL 读写分离双数据源	直接对应「数据访问优化」
三选	⑧ 跨国多时区时间一致性	数据一致性维度, 特色鲜明
加分	⑩ MDM 主数据统一	数据治理维度

难点示例 (⑧ 多时区, 非常出彩)：

系统服务日本本部（源库运行于 JST），而容器环境为 UTC、开发终端为 UTC+8。三套时区并存时，预约截止时间、邮件发送时刻等跨时区比较极易出错，后果是「提前发送」或「漏发」——对旅游业务属于严重事故。我的处理原则是**时间口径单一化**：存储层统一使用 UTC；展示层按业务方时区（JST）转换；转换动作集中到 `FormatUtils` 等工具类，禁止在业务代码中散落直接取当前时间的调用。批处理侧更彻底——`executionBaseDate`、`acquisitionRange` 等作为 `JobParameter` 显式传入，不依赖系统时钟，使同一参数的重跑结果完全可复现。◆ 补充：改造前时区相关缺陷数量与改造后的下降情况。

第 5 段 • 总结与不足 (约 200 字)

目前该数据架构已稳定支撑【◆ 数据量】，DynamoDB 侧查询延迟稳定在【◆ 数值】。

不足之处：

一是混合存储引入了运维复杂度，多套存储的备份与容量规划需要分别管理；

二是跨存储的最终一致链路缺少统一的延迟监控，异常时的排查路径较长。

后续计划引入统一的数据链路追踪与一致性校验作业。

通过本项目我认识到，数据架构设计没有“最好的存储”，

只有“与数据访问特征最匹配的存储”。

四、本项目填充点速查

论文要素	直接可写的内容
关系库	Aurora MySQL, 79 张表 (26 <code>M*</code> 主数据 + 53 <code>T*</code> 业务)，54 个 Mapper
NoSQL	DynamoDB 5 张表: travel / document / parental / sequence / message_thread
单表多实体	<code>travel</code> 表: 分区键 <code>travel_no</code> + 排序键 <code>data_type</code> + <code>json_data</code>
序列号	<code>sequence</code> 表 (<code>seq_key</code> / <code>seq_no</code>)，分布式唯一编号
对象存储	S3, 区域 ap-northeast-1
ORM	MyBatis 3.0.5 (强制锁定) ; DynamoDB Enhanced Client
读写分离	reader/writer 双数据源, 包路径隔离 Mapper
事务隔离	<code>TRANSACTION_READ_COMMITTED</code>
批量游标	<code>mybatis.defaultFetchSize = 200</code>
扫描边界	DynamoDB <code>scan-limit = 10000</code>
并发控制	乐观锁重试 4 次 / 间隔 500ms (<code>@EnableRetry</code>)
数据同步	DynamoDB Stream / SQS → 14 个 Lambda → MySQL
CSV 导出	SuperCSV 注解, ACT/AIR/HTL 三类成本文件
主数据同步	<code>ImportMdmJob</code> 定时从 MDM API 拉取

论文要素	直接可写的内容
实体组织	独立 source set <code>src/main/entity</code>
通用查询	<code>GenericQueryExecutionJob</code> + <code>MGenericQuery*</code> 表 (元数据驱动)
环境隔离	表名带环境前缀, 「1 账户 1 集群」

💡「元数据驱动的通用查询」是个隐藏亮点：`GenericQueryExecutionJob` 配合 `MGenericQuery*` 表，把查询逻辑外置为元数据，新增查询无需改代码。写「数据治理」「架构可扩展性」时非常好用。

五、可用难点素材（按适配度排序）

- 1. ⑤ **DynamoDB 单表多实体 + 混合存储** ★★ ★ —— 本主题首选
- 2. ④ **MySQL 读写分离双数据源** ★★ ★ —— 数据访问优化
- 3. ⑧ **跨国多时区一致性** ★★ ★ —— 特色，强烈建议写
- 4. ⑥ **Serverless 数据同步与最终一致性** ★★ ★ —— 一致性保障
- 5. ⑩ **MDM 主数据统一** ★★ —— 数据治理

六、踩坑提醒

坑	后果	正确做法
只说「用了 MySQL 和 DynamoDB」	没有选型逻辑	必须写 为什么 ：哪类数据适合哪种存储，判据是什么
把 NoSQL 说成「性能更好」	外行表述	正确表述是「schema 灵活、可按需扩缩、单分区内查询低延迟」
回避一致性代价	关键失分	明确写出「牺牲强一致换弹性」，并说明为何业务可接受
编造分库分表	高风险	BASE HOME 未见分库分表证据 ；改写「读写分离双数据源」，同样有分量
编造 Redis 缓存数据	高风险	项目 Redis 仅用于 Session ，不要写成业务数据缓存
忽略数据治理	漏考点	补上主数据同步、消息 ID 规范、CSV 导出规范等治理手段
大谈数据湖/湖仓一体	无支撑	本项目无数据湖，不要硬套

⚠️ **两个技术真实性红线**：① 本项目 **Redis 用于 Session 管理**（`spring-session-data-redis`），未见作为业务数据缓存的证据——写缓存架构时请谨慎，或如实说明「缓存主要用于会话」。② 本项目 **未见分库分表（Sharding）** 证据，数据量级应描述为「通过读写分离与存储选型应对」，而非「分库分表」。

配套：`04-高并发与高可用架构.md` · `08-批处理架构设计.md`（读写分离细节可复用）